Highlights

A Formalized Foundation for Research in Untyped Lambda Calculus

Chris Anto Fröschl,Prof. Matthias Güdemann

- First machine-checked formalization of Crole's five equivalent definitions of alpha equivalence.
- An isomorphism, formalized in Lean 4, between a de Bruijn representation and the quotient of named terms by alpha equivalence, following Norrish's proof pearl.
- A roadmap toward a formalized foundation for untyped lambda calculus research, including further binding representations and a "formalized Barendregt".

A Formalized Foundation for Research in Untyped Lambda Calculus

Chris Anto Fröschl[a,*], Prof. Matthias Güdemann[a,*]

[a]Munich University of Applied Sciences, Department of Computer Science and Mathematics, Lothstraße 34, Munich, D-80335, Bavaria, Germany

ARTICLE INFO

Keywords:
α-equivalence
Atom
λ-expression
Permutation action
Renaming
Variable binding
de Bruijn indices
formal verification
proof assistant
Lean
2020 MSC: 03B70
68V15
68V20

ABSTRACT

TODO Intro CSLib, Crole 5 equivs, dB = QAEQUIV and what it builds on Derived results from it, "Formalized Barendregt" Perhaps even benchmark like lambda n ways?

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

1. Introduction

Variable binding is the recurring technical obstacle in any formalization touching the lambda calculus: essentially every representation choice (named terms modulo alpha equivalence, de Bruijn indices, locally nameless terms, higher-order abstract syntax, nominal sets, …) trades one set of proof obligations for another, and results proved for one representation rarely transfer automatically to the others. This has led to a large but fragmented body of formalization work, often redone from scratch for each new proof assistant or each new project, with little of it packaged as a reusable library.

CSLib is an effort to build such a reusable, foundational library for computer science results in the Lean 4 theorem prover, in the spirit of Mathlib for mathematics [3]. This paper reports on two contributions to CSLib's treatment of the untyped lambda calculus:

1. a machine-checked formalization of Crole's result that five common literature definitions of alpha equivalence on named lambda terms are equivalent [4], which to our knowledge had not previously been formalized in a proof assistant; and
2. an isomorphism, following Norrish's proof pearl [10], between the quotient of named terms by a chosen alpha equivalence and a de Bruijn representation, connecting the named and nameless ends of CSLib's binding infrastructure.

We present both results as part of a longer-term plan: to grow CSLib into an idiomatic, well-reviewed, and maintained foundation that other researchers can build on directly, rather than reformalizing lambda calculus basics for each new project. Section 6 sketches this plan in more detail, including further equivalences in the style of Crole's for SK combinator calculus and other binding representations, and a longer-term goal of formalizing a substantial part of Barendregt's reference treatment of the untyped lambda calculus [2].

Outline. Section 2 introduces CSLib and its current named representation of the untyped lambda calculus. Section 3 presents the formalization of Crole's alpha-equivalence equalities. Section 4 presents the isomorphism with the de Bruijn representation. Section 5 discusses related formalization efforts, Section 6 lays out planned extensions, and Section 7 concludes.

*Corresponding author
chris.froeschl@hm.edu (C.A. Fröschl); matthias.guedemann@hm.edu (Prof.M. Güdemann)
ORCID(s): 0009-0006-3562-5395 (C.A. Fröschl); 0000-0002-1002-6023 (Prof.M. Güdemann)

2. CSLib and its named representation of the lambda calculus

2.1. CSLib

CSLib is a community-maintained library of foundational computer science formalizations for the Lean 4 theorem prover, built on top of Mathlib. [3] Its goal is to provide idiomatic, well-reviewed, and actively maintained formalizations of standard results, so that later research projects can build directly on a shared, trusted base rather than re-deriving basic definitions and lemmas from scratch. The work reported in this paper concerns CSLib's treatment of the untyped lambda calculus, and in particular its representation(s) of variable binding.

2.2. Named terms

CSLib currently represents untyped lambda terms in a standard named style: variables are drawn from an infinite set of atoms, and terms are built from variables, application, and abstraction. Our notation follows Crole [4], except that we write the concrete syntax of the lambda calculus where Crole writes $P(E_1, E_2)$ and $B([a]E)$ for a generic binary constructor and a generic binder; Crole notes that his proofs are insensitive to this choice [4, Section 2].

Definition 2.1 (Named lambda terms)**.** Let $\mathbb{A}$ be a set of atoms. The set Λ of named lambda terms is generated by

$$M \ ::= \ a \mid M\ M \mid \lambda a.\, M \qquad (a \in \mathbb{A}).$$

We write $\mathrm{occ}(M)$ for the finite set of all atoms occurring in M, including binding occurrences, and $\mathrm{free}(M)$ for the finite set of atoms occurring free in M, and abbreviate

$$a \not\triangleright M \iff a \notin \mathrm{occ}(M) \qquad \text{and} \qquad a \# M \iff a \notin \mathrm{free}(M).$$

In the second case we say that a is *fresh* for M. Both notations extend to lists, so that $z \not\triangleright a, b, M, M'$ means that z differs from a and b and occurs in neither M nor M'.

Crole assumes $\mathbb{A}$ to be countable, with equality of atoms as its only operation. In CSLib, $\mathbb{A}$ is an arbitrary type with decidable equality that comes with a function $\mathtt{fresh}$ from finite sets of atoms to atoms such that $\mathtt{fresh}(S) \notin S$; in particular $\mathbb{A}$ is infinite, and an atom avoiding any given finite set can be computed.

Two operations replace atoms by atoms. The first is purely textual and is the one CSLib's baseline alpha equivalence is phrased with.

Definition 2.2 (Renaming)**.** $M\langle a \mapsto b\rangle$ is M with every occurrence of a, free, bound or binding, replaced by b.

The second is capture-avoiding substitution, which Crole writes $E\{a'/a\}$ for the substitution of an atom and which we write $M[a := N]$ for the substitution of an arbitrary term.

Definition 2.3 (Capture-avoiding substitution)**.**

$$b[a := N] = \begin{cases} N & \text{if } b = a \\ b & \text{otherwise} \end{cases}$$

$$(M_1\ M_2)[a := N] = M_1[a := N]\ M_2[a := N]$$

$$(\lambda b.\, M)[a := N] = \begin{cases} \lambda b.\, M & \text{if } b = a \\ \lambda b.\, M[a := N] & \text{if } b \neq a \text{ and } b \# N \\ \lambda c.\, M\langle b \mapsto c\rangle[a := N] & \text{otherwise, where } c = \mathtt{fresh}(\mathrm{occ}(M) \cup \mathrm{occ}(N) \cup \{a, b\}) \end{cases}$$

This is substitution in the style of Curry, which renames a binder only when it has to and is defined on all terms [2, Definition C.1]. The formulation with a function choosing the new atom, and the proof that it is well defined by induction on the size of the term, are [6, Definition 5.13 and Lemma 5.14]. These definitions, a notion of alpha equivalence (Definition 3.2 below) and its basic theory, including the substitution lemma [2, 2.1.16], are due to Yin and Montesi[1] and are the starting point of our work: the alpha equivalence of CSLib is the reference point against which the other definitions are shown equivalent in Section 3.

[1] https://github.com/leanprover/cslib/pull/458

3. Characterizations of alpha equivalence

The literature defines alpha equivalence of named terms in several ways: as the congruence generated by an axiom for renaming a bound atom, with one of two side conditions on the new atom, or, following Gabbay and Pitts [7], by a syntax-directed system of rules in which the two bound atoms are swapped with a third. That these definitions agree is folklore, but Crole [4] observes that the proof sketches in the literature are sometimes incorrect, and gives a detailed proof for five such relations. This section presents his result as it is formalized in CSLib. We keep Crole's names for the relations, the rules and the side conditions; each statement names its counterpart in [4], and a raised LEAN links to its Lean source.

3.1. Swapping

Definition 3.1 (Permutation action [4, Section 2])**.** LEAN For a permutation π of $\mathbb{A}$, the term $\pi \cdot M$ is defined by

$$\pi \cdot a = \pi(a) \qquad \pi \cdot (M_1\ M_2) = (\pi \cdot M_1)\,(\pi \cdot M_2) \qquad \pi \cdot (\lambda a.\ M) = \lambda \pi(a).\ \pi \cdot M.$$

For the transposition $(a\ b)$ we call $(a\ b) \cdot M$ the *swap* LEAN of a and b in M. The *agreement set* LEAN of two permutations is $\mathrm{AS}(\pi, \pi') = \{\, a \in \mathbb{A} \mid \pi(a) = \pi'(a) \,\}$.

Unlike renaming (Definition 2.2), a swap is a bijection on terms and never identifies two atoms. The two operations agree when the new atom is unused: if $z \not\in M$ then $(z\ a) \cdot M = M\langle a \mapsto z \rangle$ LEAN.

3.2. The five relations

The first three relations are syntax-directed and differ only in the side condition on the atom z that replaces the two bound atoms.

Definition 3.2 ([4, Definitions 3.1–3.3])**.** The relation $\sim_{\mathrm{p}}$ LEAN on Λ is inductively defined by

$$\frac{}{a \sim_{\mathrm{p}} a}\ \textsc{Atom} \qquad \frac{M_1 \sim_{\mathrm{p}} M_1' \quad M_2 \sim_{\mathrm{p}} M_2'}{M_1\ M_2 \sim_{\mathrm{p}} M_1'\ M_2'}\ \textsc{Pcg} \qquad \frac{(z\ a) \cdot M \sim_{\mathrm{p}} (z\ b) \cdot M'}{\lambda a.\ M \sim_{\mathrm{p}} \lambda b.\ M'}\ \textsc{Pi}\ [z \not\in a, b, M, M'].$$

The relations $\sim_{\mathrm{p}\#}$ LEAN and $\sim_{\mathrm{p}}^{1}$ LEAN are defined by the same rules, with the side condition of PI replaced by

$$[z \# a, b, M, M'] \qquad \text{and} \qquad [z \not\in M, M'],$$

respectively; we call the resulting rules PI# and PI1.

The relation $\sim_{\mathrm{p}}$ follows Gabbay and Pitts [7] and is CSLib's alpha equivalence $=_\alpha$. The remaining two relations are generated by the renaming axiom and are congruences by definition.

Definition 3.3 ([4, Definitions 3.4 and 3.5])**.** The relation $\sim_{\mathrm{r}}$ LEAN on Λ is inductively defined by

$$\frac{}{M \sim_{\mathrm{r}} M}\ \textsc{Ref} \qquad \frac{M_1 \sim_{\mathrm{r}} M_2}{M_2 \sim_{\mathrm{r}} M_1}\ \textsc{Sym} \qquad \frac{M_1 \sim_{\mathrm{r}} M_2 \quad M_2 \sim_{\mathrm{r}} M_3}{M_1 \sim_{\mathrm{r}} M_3}\ \textsc{Trs}$$

$$\frac{M_1 \sim_{\mathrm{r}} M_1' \quad M_2 \sim_{\mathrm{r}} M_2'}{M_1\ M_2 \sim_{\mathrm{r}} M_1'\ M_2'}\ \textsc{Pcg} \qquad \frac{M \sim_{\mathrm{r}} M'}{\lambda a.\ M \sim_{\mathrm{r}} \lambda a.\ M'}\ \textsc{Bcg} \qquad \frac{}{\lambda a.\ M \sim_{\mathrm{r}} \lambda a'.\ M[a := a']}\ \alpha\ [a' \not\in a, M].$$

The relation $\sim_{\mathrm{r}\#}$ LEAN is defined by the same rules, with the side condition of α replaced by $[a' \# a, M]$; we call the resulting rule α#.

The relation $\sim_{\mathrm{r}}$ corresponds to the congruence $\equiv_\alpha$ of Barendregt [2, Definition 2.1.11], and $\sim_{\mathrm{r}\#}$ to that of Hindley and Seldin [8, Definition 1.17].

3.3. Properties of $\sim_p$

Since $\sim_p$ has no rules for reflexivity, symmetry and transitivity, these have to be proved. They were already available in CSLib, together with the two other basic facts below; the corresponding statements for $\equiv_\alpha$ are [8, Lemmas 1.19 and 1.21], proved in [8, Appendix A1].

Lemma 3.4. *The relation $\sim_p$ is reflexive, symmetric and transitive. Moreover, if $M \sim_p M'$ and $N \sim_p N'$ then $\mathrm{free}(M) = \mathrm{free}(M')$ and $M[a := N] \sim_p M'[a := N']$.*

The next two lemmas say that $\sim_p$ is invariant under swapping, an instance of the equivariance principle of nominal techniques [6, Theorem 4.4], and that the action of a permutation on a term depends, up to $\sim_p$, only on its values on the free atoms.

Lemma 3.5 ([4, Lemma 6.1])**.** *If $M \sim_p M'$ then $(u\ v) \cdot M \sim_p (u\ v) \cdot M'$, for all atoms u, v.*

Lemma 3.6 ([4, Lemma 6.2])**.** *Let π, π' be permutations of $\mathbb{A}$ and M a term.*

1. *If $\mathrm{occ}(M) \subseteq \mathrm{AS}(\pi, \pi')$ then $\pi \cdot M = \pi' \cdot M$.*
2. *If $\mathrm{free}(M) \subseteq \mathrm{AS}(\pi, \pi')$ then $\pi \cdot M \sim_p \pi' \cdot M$.*

The bridge between the two families of relations is that swapping with an unused atom is, up to alpha equivalence, substitution. For the renaming relations this is [4, Lemmas 6.3 and 6.4]. Gabbay proves it as a syntactic identity under the additional hypothesis that a is not bound in M [6, Lemma 5.16].

Lemma 3.7. *If $z \not\triangleright M$ then $(z\ a) \cdot M \sim_r M[a := z]$ and $(z\ a) \cdot M \sim_{r\#} M[a := z]$.*

For $\sim_p$ the corresponding statement compares two swaps.

Lemma 3.8 ([4, Lemma 6.5])**.** *Let $z_1, a_1, a'_1, \ldots, z_n, a_n, a'_n$ be pairwise distinct atoms and let M be a term with $z_i \# M$ and $a'_i \# M$ for all i. Then*

$$(z_1\ a_1) \cdot \ldots \cdot (z_n\ a_n) \cdot M \ \sim_p \ (z_1\ a'_1) \cdot \ldots \cdot (z_n\ a'_n) \cdot M[a_1 := a'_1] \ldots [a_n := a'_n].$$

Only the case $n = 1$ is used below. Crole assumes $z_i \not\triangleright M$; freshness suffices, because a bound occurrence of z_i can be renamed away first:

Lemma 3.9 ([4, Lemma 6.6])**.** *If $a \# M$ then there is a term $\hat{M}$ with $a \not\triangleright \hat{M}$ and $\hat{M} \sim_r M$, and likewise with $\sim_p$ in place of $\sim_r$.*

This lemma justifies Barendregt's variable convention [2, 2.1.13]; for $\equiv_\alpha$ it is [8, Lemma A1.5(g)]. The last lemma is the counterpart of the second half of Lemma 3.4 for the renaming relation, and a special case of [2, Proposition C.2] and [8, Lemma A1.10].

Lemma 3.10 ([4, Lemma 6.7])**.** *If $M \sim_r M'$ then $M[b := a] \sim_r M'[b := a]$, for all atoms a, b.*

3.4. The characterization theorem

Theorem 3.11 (Crole [4, Theorems 4.1, 4.2, 4.4–4.6])**.** *For all terms $M, N \in \Lambda$ the following are equivalent:*

(i) $M \sim_p N$;
(ii) $M \sim_{p\#} N$;
(iii) $M \sim_p^1 N$;
(iv) $M \sim_r N$;
(v) $M \sim_{r\#} N$.

Proof. Each of (ii)–(v) is shown equivalent to (i), by showing that each of the two relations is closed under the rules of the other. Only the rules for abstractions need an argument.

(i)⇔(ii). Non-occurrence implies freshness, so PI is an instance of PI#. Conversely, let $(u\ a) \cdot M \sim_p (u\ b) \cdot M'$ with $u \# a, b, M, M'$, and choose $z \not\triangleright u, a, b, M, M'$. By Lemma 3.5, $(z\ u) \cdot (u\ a) \cdot M \sim_p (z\ u) \cdot (u\ b) \cdot M'$. The

permutations $(z\ u)\circ(u\ a)$ and $(z\ a)$ agree on $\mathrm{free}(M)$, so the left-hand side is $\sim_{\mathrm{p}}$-related to $(z\ a)\cdot M$ by Lemma 3.6, and likewise for the right-hand side. Hence $(z\ a)\cdot M \sim_{\mathrm{p}} (z\ b)\cdot M'$, and PI applies.

(i)⇔(iii). The rule PI1 is more general than PI only in allowing $z \in \{a, b\}$. If $z = a$, its premise reads $M \sim_{\mathrm{p}} (a\ b)\cdot M'$ with $a \not\# M'$. For $z' \not\# a, b, M, M'$, Lemma 3.5 and the first part of Lemma 3.6 give $(z'\ a)\cdot M \sim_{\mathrm{p}} (z'\ b)\cdot M'$. The case $z = b$ is symmetric.

(i)⇔(iv). For PI, let $(z\ a)\cdot M \sim_{\mathrm{r}} (z\ b)\cdot M'$ with $z \not\# a, b, M, M'$. Lemma 3.7 gives $M[a := z] \sim_{\mathrm{r}} M'[b := z]$, and then

$$\lambda a.\, M \ \sim_{\mathrm{r}}\ \lambda z.\, M[a := z] \ \sim_{\mathrm{r}}\ \lambda z.\, M'[b := z] \ \sim_{\mathrm{r}}\ \lambda b.\, M'$$

by α, BCG and α. Conversely, REF, SYM and TRS hold for $\sim_{\mathrm{p}}$ by Lemma 3.4, and BCG by Lemma 3.5 and PI. For α, let $a' \not\# a, M$ and choose $z \not\# a, a', M, M[a := a']$. Then $(z\ a)\cdot M \sim_{\mathrm{p}} (z\ a')\cdot M[a := a']$ by Lemma 3.8, which is the premise of PI for $\lambda a.\, M \sim_{\mathrm{p}} \lambda a'.\, M[a := a']$.

(i)⇔(v). As in the previous case: Lemma 3.7 covers $\sim_{\mathrm{r}\#}$, and Lemma 3.8 only requires $a' \# M$. □

In the last case Crole's argument differs from ours. He proves the rule α by an induction on M that fails for $\alpha\#$, where the substitution $M[a := a']$ may have to rename a binder, and introduces Lemma 3.8 for general n to repair it. Proving the case $n = 1$ under the freshness hypothesis serves both rules.

The equivalence of (iv) and (v) follows from the theorem, but it is a statement about renaming alone, and Crole also proves it without recourse to swapping [4, Theorem 4.6] LEAN. Since non-occurrence implies freshness, α is an instance of $\alpha\#$. Conversely, let $a' \# a, M$. By Lemma 3.9 there is $\hat{M} \sim_{\mathrm{r}} M$ with $a' \not\# \hat{M}$, and

$$\lambda a.\, M \ \sim_{\mathrm{r}}\ \lambda a.\, \hat{M} \ \sim_{\mathrm{r}}\ \lambda a'.\, \hat{M}[a := a'] \ \sim_{\mathrm{r}}\ \lambda a'.\, M[a := a']$$

by BCG, by α, and by Lemma 3.10 with BCG. Hindley and Seldin relate $\equiv_\alpha$ to a restricted renaming step by the same argument [8, Lemmas A1.7 and A1.8].

The side condition of PI cannot be weakened much further.

Proposition 3.12 ([4, Proposition 4.3])**.** LEAN *Let* $\sim_{\mathrm{p}}^{2}$ LEAN *and* $\sim_{\mathrm{p}}^{3}$ LEAN *be defined as* $\sim_{\mathrm{p}}$*, with the side condition of* PI *replaced by* $[z \not\# a, b]$ *and by no condition, respectively. Then* $\sim_{\mathrm{p}}^{2} \neq \sim_{\mathrm{p}}$ *and* $\sim_{\mathrm{p}}^{3} \neq \sim_{\mathrm{p}}$*.*

Proof. For distinct atoms a, b, z we have $\lambda a.\, a\ z \sim_{\mathrm{p}}^{2} \lambda b.\, b\ a$ and $\lambda a.\, b\ a \sim_{\mathrm{p}}^{3} \lambda b.\, a\ b$, but in both cases the two terms have different free atoms, so they are not related by $\sim_{\mathrm{p}}$ (Lemma 3.4). □

3.5. The formalization

The formalization follows [4] closely, and we found no errors in it. To our knowledge the result had not been mechanized before; Crole mentions an Isabelle/HOL development in progress [4, Section 7], of which we could find no published account. Our contribution is the four relations other than $\sim_{\mathrm{p}}$, Theorem 3.11, Proposition 3.12, and Lemmas 3.5–3.10. The definition of $\sim_{\mathrm{p}}$ and Lemma 3.4 belong to the named representation that this work builds on (Section 2.2). The first part of the development is under review for inclusion in CSLib[2]; the remainder is to follow in the same way.

What a formalization has to add to [4] are the proofs that the paper leaves to the reader or to the literature. Table 1 lists them, with a place where the argument can be consulted on paper. None of them held a surprise.

Three statements differ from their counterparts in [4]. The rules PI and PI1 are stated in CSLib with renaming (Definition 2.2) in place of the swap, which is the same under their side conditions. Lemma 3.6 is proved for arbitrary bijections on $\mathbb{A}$, where Crole assumes finite support; the difference is immaterial for terms [6, Remark 2.25]. And Lemma 3.8 has the weaker hypothesis discussed above. Crole's second contribution, alpha equivalence of program contexts [4, Section 5], is not covered.

4. An isomorphism with de Bruijn terms

De Bruijn indices remove the need for alpha equivalence altogether by replacing bound variable names with numerals counting binder depth. Norrish and Vestergaard [10] give a direct, mechanized proof that the pure de Bruijn

[2] https://github.com/leanprover/cslib/pull/741

Table 1
Proofs omitted or only sketched in [4].

Statement in [4]	Treatment there	See
Existence of substitution (Theorem A.1)	deferred	[6, Definition 5.13, Lemma 5.14]
Lemma 6.1	representative cases only	Lemma 3.5; [6, Theorem 4.4]
Lemma 6.2, part 1	omitted, as for part 2	Lemma 3.6
Lemma 6.5	details omitted	Lemma 3.8
Lemma 6.6	sketched as well known	[8, Lemma A1.5(g)]
Lemma 6.7	deferred as well known	[8, Lemma A1.10]; [5, pp. 96–103]

calculus is isomorphic to the alpha-quotiented named calculus, so that a result proved on de Bruijn terms is a result about lambda terms in the ordinary sense. We formalize this isomorphism for the named terms and the de Bruijn terms of CSLib. By Theorem 3.11 it does not matter which of the five alpha equivalences is quotiented by; the formalization uses $\sim_{\mathrm{p}}$.

We keep the notation of [10] on the de Bruijn side, where t, u, s range over de Bruijn terms and i, j, k, n over indices. As in [10], M and N range over alpha-equivalence classes in this section.

4.1. De Bruijn terms

Definition 4.1 (De Bruijn terms [10, Section 2.2]). The set dB of de Bruijn terms is generated by

$$t \ ::= \ \mathsf{dV}\, i \mid \mathsf{dAPP}\, t\, t \mid \mathsf{dABS}\, t \qquad (i \in \mathbb{N}).$$

Lifting and substitution are defined as in Nipkow's formalization [9]:

$$\begin{aligned}
\mathsf{lift}\,(\mathsf{dV}\, i)\, k &= \text{if } i < k \text{ then } \mathsf{dV}\, i \text{ else } \mathsf{dV}\,(i+1)\\
\mathsf{lift}\,(\mathsf{dAPP}\, t\, u)\, k &= \mathsf{dAPP}\,(\mathsf{lift}\, t\, k)\,(\mathsf{lift}\, u\, k)\\
\mathsf{lift}\,(\mathsf{dABS}\, t)\, k &= \mathsf{dABS}\,(\mathsf{lift}\, t\,(k+1))
\end{aligned}$$

$$\begin{aligned}
\mathsf{nsub}\, s\, k\,(\mathsf{dV}\, i) &= \text{if } k < i \text{ then } \mathsf{dV}\,(i-1) \text{ else if } i = k \text{ then } s \text{ else } \mathsf{dV}\, i\\
\mathsf{nsub}\, s\, k\,(\mathsf{dAPP}\, t\, u) &= \mathsf{dAPP}\,(\mathsf{nsub}\, s\, k\, t)\,(\mathsf{nsub}\, s\, k\, u)\\
\mathsf{nsub}\, s\, k\,(\mathsf{dABS}\, t) &= \mathsf{dABS}\,(\mathsf{nsub}\,(\mathsf{lift}\, s\, 0)\,(k+1)\, t)
\end{aligned}$$

The function nsub substitutes for the index k and at the same time decrements all larger free indices, which is what β-contraction needs when a binder disappears.

Definition 4.2 ([10, Definition 1]). The relation $\to_{\mathrm{d}}$ on dB is the least relation satisfying

$$\frac{}{\mathsf{dAPP}\,(\mathsf{dABS}\, t)\, u \to_{\mathrm{d}} \mathsf{nsub}\, u\, 0\, t} \qquad \frac{t \to_{\mathrm{d}} u}{\mathsf{dABS}\, t \to_{\mathrm{d}} \mathsf{dABS}\, u}$$

$$\frac{t \to_{\mathrm{d}} u}{\mathsf{dAPP}\, t\, s \to_{\mathrm{d}} \mathsf{dAPP}\, u\, s} \qquad \frac{t \to_{\mathrm{d}} u}{\mathsf{dAPP}\, s\, t \to_{\mathrm{d}} \mathsf{dAPP}\, s\, u}\,.$$

4.2. The alpha-quotiented calculus

Definition 4.3 ([10, Section 2.1]). Λ_α is the quotient of Λ by $\sim_{\mathrm{p}}$. We write $[M]_\alpha$ for the class of a term $M \in \Lambda$. Application, abstraction and substitution are lifted from Λ to Λ_α:

$$[M]_\alpha\,[N]_\alpha = [M\ N]_\alpha \qquad \lambda a.\,[M]_\alpha = [\lambda a.\, M]_\alpha \qquad [M]_\alpha[a := [N]_\alpha] = [M[a := N]]_\alpha.$$

These operations are well defined because $\sim_{\mathrm{p}}$ is a congruence and is preserved by substitution. Abstraction on Λ_α is not injective, so Λ_α is not a free algebra.

Definition 4.4 (β-reduction on Λ_α)**.** The relation $\to_\beta$ on Λ_α is the least relation satisfying

$$\frac{}{(\lambda a.\, M)\, N \to_\beta M[a := N]} \qquad \frac{M \to_\beta N}{\lambda a.\, M \to_\beta \lambda a.\, N}$$

$$\frac{M \to_\beta N}{M\, Z \to_\beta N\, Z} \qquad \frac{M \to_\beta N}{Z\, M \to_\beta Z\, N}\,.$$

This is the system $\lambda_\alpha^{\mathrm{N}}$ of [10]: substitution and reduction are defined on the quotient itself and not on raw terms.

Two systems $\langle X, \to_X\rangle$ and $\langle Y, \to_Y\rangle$ are *isomorphic* if there is a bijection $f : X \to Y$ with $x_1 \to_X x_2 \iff f(x_1) \to_Y f(x_2)$ for all x_1, x_2 [10, Section 2.3].

4.3. An intermediate formalism

The proof goes through a second presentation dB′ of the same type dB that is built to resemble Λ_α. It replaces nsub by a substitution that leaves the other indices alone, and dABS by an abstraction over a chosen index.

Definition 4.5 ([10, Definitions 4 and 5])**.**

$$\begin{aligned}
\mathsf{sub}\; s\; k\; (\mathsf{dV}\; i) &= \text{if } i = k \text{ then } s \text{ else } \mathsf{dV}\; i\\
\mathsf{sub}\; s\; k\; (\mathsf{dAPP}\; t\; u) &= \mathsf{dAPP}\; (\mathsf{sub}\; s\; k\; t)\; (\mathsf{sub}\; s\; k\; u)\\
\mathsf{sub}\; s\; k\; (\mathsf{dABS}\; t) &= \mathsf{dABS}\; (\mathsf{sub}\; (\mathsf{lift}\; s\; 0)\; (k+1)\; t)\\
\mathsf{dLAM}\; i\; t &= \mathsf{dABS}\; (\mathsf{sub}\; (\mathsf{dV}\; 0)\; (i+1)\; (\mathsf{lift}\; t\; 0))
\end{aligned}$$

Like abstraction on Λ_α, dLAM is not injective. Every dABS term is a dLAM term over any index that is not free in it [10, Lemma 6].

Definition 4.6 ([10, Definition 7])**.** The relation $\to_{\mathrm{d}'}$ on dB is the least relation satisfying

$$\frac{}{\mathsf{dAPP}\; (\mathsf{dLAM}\; i\; t)\; u \to_{\mathrm{d}'} \mathsf{sub}\; u\; i\; t} \qquad \frac{t \to_{\mathrm{d}'} u}{\mathsf{dLAM}\; i\; t \to_{\mathrm{d}'} \mathsf{dLAM}\; i\; u}$$

$$\frac{t \to_{\mathrm{d}'} u}{\mathsf{dAPP}\; t\; s \to_{\mathrm{d}'} \mathsf{dAPP}\; u\; s} \qquad \frac{t \to_{\mathrm{d}'} u}{\mathsf{dAPP}\; s\; t \to_{\mathrm{d}'} \mathsf{dAPP}\; s\; u}\,.$$

4.4. The bijection

Fix a bijection between $\mathbb{A}$ and $\mathbb{N}$, written with a hat in both directions as in [10]: $\hat{a}$ is the number of the atom a and $\hat{\imath}$ the atom with number i.

Theorem 4.7 ([10, Theorem 11])**.** *There is a bijection $f : \Lambda_\alpha \to \mathrm{dB}$ with*

$$f([a]_\alpha) = \mathsf{dV}\; \hat{a} \qquad f(M\; N) = \mathsf{dAPP}\; (f\; M)\; (f\; N) \qquad f(\lambda a.\, M) = \mathsf{dLAM}\; \hat{a}\; (f\; M). \tag{1}$$

Since Λ_α is not a free algebra, the equations (1) do not define a function by themselves. Norrish and Vestergaard obtain f from Pitts' recursion principle for Λ_α [11], for which dB has to be shown to be a nominal set. The formalization takes a more elementary route: the equations (1) define a function on raw terms by structural recursion, this function is shown to respect $\sim_{\mathrm{p}}$, and f is its lift to the quotient.

The bijection turns substitution on Λ_α into the index-preserving substitution sub, and with it β-reduction into $\to_{\mathrm{d}'}$.

Lemma 4.8 ([10, proof of Theorem 12])**.** $f(M[a := N]) = \mathsf{sub}\; (f\; N)\; \hat{a}\; (f\; M)$.

Theorem 4.9 ([10, Theorem 12])**.** *For all $M, N \in \Lambda_\alpha$: $M \to_\beta N$ if and only if $f(M) \to_{\mathrm{d}'} f(N)$.*

4.5. The isomorphism

It remains to show that $\to_{\mathrm{d}}$ and $\to_{\mathrm{d}'}$ are the same relation on dB. Their rules for redexes are related by an identity between sub and nsub [10, Lemma 13]; the difficulty is to match the congruence rule for dABS with the one for dLAM. This is where permutations enter on the de Bruijn side. A permutation π of atoms acts on a de Bruijn term through the free indices:

$$\pi \cdot (\mathsf{dV}\, i) = \mathsf{dV}\, \widehat{\pi(i)} \qquad \pi \cdot (\mathsf{dAPP}\, t\, u) = \mathsf{dAPP}\, (\pi \cdot t)\, (\pi \cdot u) \qquad \pi \cdot (\mathsf{dABS}\, t) = \mathsf{dABS}\, (\mathsf{inc}(\pi) \cdot t),$$

where $\mathsf{inc}(\pi)$ is π shifted by one index, so that it fixes the index bound by the abstraction [10, Definition 8]. A relation R on dB is *equivariant* if $R\, (\pi \cdot t)\, (\pi \cdot u) \iff R\, t\, u$ for all π.

Theorem 4.10 ([10, Theorem 10])**.** *For every permutation π of finite support:*

$$\pi \cdot (\mathsf{sub}\, t\, j\, u) = \mathsf{sub}\, (\pi \cdot t)\, \widehat{\pi(j)}\, (\pi \cdot u)$$

$$\pi \cdot (\mathsf{dLAM}\, i\, t) = \mathsf{dLAM}\, \widehat{\pi(i)}\, (\pi \cdot t)$$

$$\pi \cdot t \to_{\mathrm{d}'} \pi \cdot u \iff t \to_{\mathrm{d}'} u$$

The key observation of [10] is that lifting is itself the action of a permutation [10, Lemma 14], and so is the substitution of a fresh index. A dABS term and its presentation as a dLAM term therefore differ by a permutation, which an equivariant relation cannot see.

Theorem 4.11 ([10, Theorem 15])**.** *Let R be an equivariant relation on* dB*. Then*

$$\big(\forall t\, u.\ R\, t\, u \Rightarrow R\, (\mathsf{dABS}\, t)\, (\mathsf{dABS}\, u)\big) \iff \big(\forall i\, t\, u.\ R\, t\, u \Rightarrow R\, (\mathsf{dLAM}\, i\, t)\, (\mathsf{dLAM}\, i\, u)\big).$$

Theorem 4.11 applies to $\to_{\mathrm{d}'}$ by Theorem 4.10. To apply it to $\to_{\mathrm{d}}$ one needs equivariance of $\to_{\mathrm{d}}$, and hence the behaviour of nsub under a permutation [10, Lemma 17].

Theorem 4.12 ([10, Theorem 19])**.** *For all $t, u \in$ dB: $t \to_{\mathrm{d}'} u$ if and only if $t \to_{\mathrm{d}} u$.*

Theorem 4.13 ([10, Theorem 20])**.** *The systems $\langle \Lambda_\alpha, \to_\beta \rangle$ and $\langle \mathrm{dB}, \to_{\mathrm{d}} \rangle$ are isomorphic, as witnessed by the bijection f of Theorem 4.7: for all $M, N \in \Lambda_\alpha$,*

$$M \to_\beta N \iff f(M) \to_{\mathrm{d}} f(N).$$

Proof. From Theorems 4.9 and 4.12. □

4.6. Using the isomorphism

An isomorphism of reduction systems transports every property that is stated in terms of the reduction relation. The de Bruijn development we build on proves the Church–Rosser theorem for $\to_{\mathrm{d}}$. With Theorem 4.13 it becomes a theorem about named terms up to alpha equivalence, with no further work on the named side.

Corollary 4.14 (Church–Rosser for Λ_α)**.** *The relation $\to_\beta$ on Λ_α is confluent: if $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$ then there is P with $N_1 \twoheadrightarrow_\beta P$ and $N_2 \twoheadrightarrow_\beta P$.*

Corollary 4.15. *If $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$ and both N_1 and N_2 are β-normal, then $N_1 = N_2$.*

4.7. Formalization notes

- *Atoms.* Sections 2 and 3 are parametric in $\mathbb{A}$. The isomorphism is stated for one concrete type of atoms, the natural numbers wrapped in a structure, for which the two hat maps are the projection and the constructor.
- *Permutations.* Norrish and Vestergaard represent a permutation as a list of pairs of strings. The formalization uses finitely supported bijections on atoms.
- *Substitution on* dB. CSLib does not define nsub by the equations of Definition 4.1, but as a composition of a lifting, the substitution sub and a lowering of indices. The equations of Definition 4.1 are proved as lemmas.
- *The bijection.* f is a bijection in the sense of an explicit pair of mutually inverse functions. Its inverse is obtained from surjectivity by choice and is not computable.

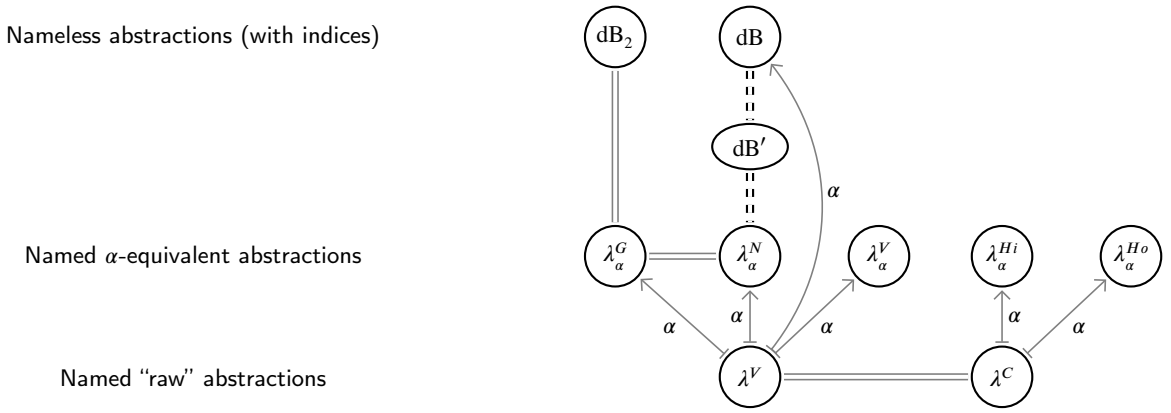


Figure 1: Formalizations of the lambda calculus and their directly proved relationships, redrawn from [10, Figure 1]. Double lines are isomorphisms and arrows are α-quotients. The dashed lines are the results of [10], which Section 4 formalizes.

5. Related work

Formalizing variable binding has a long history across proof assistants, and several general-purpose approaches exist alongside the direct, elementary style used in CSLib. Nominal techniques [12], as implemented in Nominal Isabelle [13], provide binder-generic reasoning principles via permutations and freshness, at the cost of adopting a dedicated meta-theory. Locally nameless representations [1] aim to make binder-heavy formalizations more tractable or more automatically generated from a specification, respectively. Higher-order abstract syntax sidesteps binder representation issues by delegating them to the proof assistant's own binding mechanism, again at the cost of restricting which proof techniques remain available (e.g. induction principles).

CSLib's current approach is deliberately more elementary: named terms and de Bruijn terms are both represented directly as first-order inductive types, with the relationships between representations (such as the isomorphism of Section 4) proved explicitly rather than assumed via a shared meta-level binding mechanism. This trades some proof automation for representations that are simple to state, and results that transfer cleanly between representations by construction, which fits CSLib's goal of a reusable foundation rather than a self-contained framework.

6. Future work

Norrish and Vestergaard place their isomorphism in a web of directly proved relationships between ten formalizations of the lambda calculus, reproduced in Figure 1.

The two results of this paper are intended as the first pieces of a larger, ongoing effort within CSLib to build a formalized foundation for research on untyped lambda calculus. We outline the planned directions below.

6.1. Further equivalence results

Crole-style results, showing several natural but superficially different definitions of the same notion to coincide, are not unique to alpha equivalence. We plan analogous formalizations for SK combinator calculus and other binding representations (locally nameless terms, nominal terms), each intended to slot into CSLib alongside the results of this paper and to be connected to them by explicit isomorphisms in the style of Section 4.

6.2. A formalized Barendregt

Barendregt's *The Lambda Calculus: Its Syntax and Semantics* [2] remains the standard reference for the untyped lambda calculus, and a substantial fraction of its content — confluence, standardization, the theory of reduction, combinatory completeness, and the associated notions of models — has never been formalized in a single, coherent development. We see CSLib's binding infrastructure as a natural basis for such an effort, which we tentatively refer to as a "formalized Barendregt". Beyond its intrinsic value as a reusable library, a sufficiently complete formalized Barendregt would put open or only recently resolved conjectures discussed in Barendregt's later survey work within reach of machine-checked treatment.

6.3. Maintenance and reuse

Throughout, the intent is for this work to be maintained as part of CSLib rather than as a standalone, one-off development: subject to code review, kept up to date with CSLib and Mathlib, and structured so that later projects can depend on it directly.

7. Conclusion

We have presented CSLib's current named representation of the untyped lambda calculus, a formalization of Crole's result that five notions of alpha equivalence coincide, and an isomorphism, following Norrish, between the quotient of named terms under alpha equivalence and a de Bruijn representation. Together these results connect two of the standard approaches to variable binding within a single, reusable Lean 4 library, and form the starting point for a longer-term effort to build a maintained, idiomatic foundation for research on untyped lambda calculus, including further binding representations and a formalized treatment of Barendregt's reference text.

Data availability

The Lean 4 formalization discussed in this paper is developed as part of CSLib and is publicly available at https://github.com/leanprover/cslib.

TODO consider with zenodo

References

[1] Aydemir, B., Charguéraud, A., Pierce, B.C., Pollack, R., Weirich, S., 2008. Engineering formal metatheory, in: Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Association for Computing Machinery, New York, NY, USA. p. 3–15. URL: https://doi.org/10.1145/1328438.1328443, doi:10.1145/1328438.1328443.

[2] Barendregt, H.P., 1985. The lambda calculus - its syntax and semantics. volume 103 of *Studies in logic and the foundations of mathematics*. North-Holland.

[3] Barrett, C., Chaudhuri, S., Montesi, F., Grundy, J., Kohli, P., de Moura, L., Rademaker, A., Yingchareonthawornchai, S., 2026. Cslib: The lean computer science library. URL: https://arxiv.org/abs/2602.04846, arXiv:2602.04846.

[4] Crole, R.L., 2012. Alpha equivalence equalities. Theoretical Computer Science 433, 1–19. URL: https://www.sciencedirect.com/science/article/pii/S0304397512000667, doi:10.1016/j.tcs.2012.01.030.

[5] Curry, H.B., Feys, R., 1958. Combinatory Logic. volume I. North-Holland, Amsterdam.

[6] Gabbay, M.J., 2011. Foundations of nominal techniques: logic and semantics of variables in abstract syntax. Bulletin of Symbolic Logic 17, 161–229.

[7] Gabbay, M.J., Pitts, A.M., 2002. A new approach to abstract syntax with variable binding. Formal Aspects of Computing 13, 341–363. doi:10.1007/s001650200016.

[8] Hindley, J.R., Seldin, J.P., 2008. Lambda-Calculus and Combinators: An Introduction. 2 ed., Cambridge University Press.

[9] Nipkow, T., 2001. More Church-Rosser proofs (in Isabelle/HOL). Journal of Automated Reasoning 26, 51–66.

[10] Norrish, M., Vestergaard, R., 2007. Proof pearl: De bruijn terms really do work, in: Schneider, K., Brandt, J. (Eds.), Theorem Proving in Higher Order Logics, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 207–222.

[11] Pitts, A.M., 2006. Alpha-structural recursion and induction. Journal of the ACM 53, 459–506.

[12] Pitts, A.M., 2013. Nominal Sets: Names and Symmetry in Computer Science. volume 57 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press.

[13] Urban, C., Tasson, C., 2005. Nominal techniques in isabelle/hol, in: Nieuwenhuis, R. (Ed.), Automated Deduction – CADE-20, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 38–53.